Bauhaus-Universität Weimar
Faculty of Media
Degree Programme Digital Engineering

Bauhaus-Universität Weimar

Model checking for Self-Replicating Robotic Systems

Initial Research

Master's Thesis

Dmitry Babaytsev (124065)

1. Referee: Prof. Dr. Jan Oliver Ringert
2. Referee: Prof. Dr. Unknown Yet

Submission date: June 31, 2025

Declaration

Unless otherwise indicated in the text or references, this thesis is entirely the product of my own scholarly work.

Weimar, June 31, 2025

Dmitry Babaytsev (124065)

Abstract

A short summary.

Contents

1 Introduction

Let us get started by citing [Bec+13]! So what did Beck et al. do in 2013? Maybe it is answered in chapter 1 on page 1?

You can look it up on https://en.wikipedia.org/wiki/Pythagorean_theorem.

2 Background

2.1 Formal verification overview

Formal Methods refers to mathematically rigorous techniques and tools for the specification, design and verification of software and hardware systems.
Expanding this definition Formal methods can be used for formal description of the system (specification) on some level of abstraction, program synthesis (design) according to specification and verification of a system to prove that model of system under consideration satisfies some properties.

Formal verification techniques can be thought of as comprising three parts [Mic04]:

- a framework for modelling systems, typically a description language;
- a specification language for describing the properties to be verified;
- a verification method to establish whether the description of a system satisfies the specification.

These methods and techniques of verification include:

1. SAT solving - operate on propositional logic
2. SMT solving (and tools like Z3) extend this to first-order logic, integrating theories such as arithmetic or arrays.
3. Relational model finding (e.g., Alloy)
4. Model checking (e.g., NuSMV) exhaustively explore state-space against CTL/LTL specifications.
5. Interactive theorem provers (e.g., Coq/Rocq, Isabelle/HOL)

Approaches to verification can be classified according to the following criteria [Mic04] pages 172-173:
Proof-based vs. model-based
In a proof-based approach, the system description is a set of formulas Γ (in a suitable logic) and the specification is another formula ϕ. The verification method consists of

trying to find a proof that $\Gamma \vdash \phi$. In a model-based approach, the system is represented by a model M for an appropriate logic. The specification is again represented by a formula ϕ and the verification method consists of computing whether a model M satisfies ϕ ($M \models \phi$). This computation is usually automatic for finite models. Logical proof systems are often sound and complete, meaning that $\Gamma \vdash \phi$ (provability) holds if, and only if, $\Gamma \models \phi$(semantic entailment) holds, where the latter is defined as follows: for all models M, if for all $\psi \in \Gamma$ we have $M \models \psi$, then $M \models \phi$. Thus, we see that the model-based approach is potentially simpler than the proof-based approach, for it is based on a single model M rather than a possibly infinite class of them.

Degree of automation.
Approaches differ on how automatic the method is; the extremes are fully automatic and fully manual. Many of the computer-assisted techniques are somewhere in the middle.

Full- vs. property-verification.
The specification may describe a single property of the system, or it may describe its full behavior. The latter is typically expensive to verify. Intended domain of application, which may be hardware or software; sequential or concurrent; reactive or terminating; etc. A reactive system is one which reacts to its environment and is not meant to terminate (e.g., operating systems, embedded systems and computer hardware).

Pre- vs. post-development.
Verification is of greater advantage if introduced early in the course of system development, because errors caught earlier in the production cycle are less costly to rectify. (It is alleged that Intel lost millions of dollars by releasing their Pentium chip with the FDIV error.

2.1.1 SAT solvers

Propositional logic - consider propositions and relations between them and constructions of argument based on them.

Propositions - facts or statements about the world that can be true or false. Also defined as atomic propositions or Atoms. Some propositions can be described in natural language as atomic or indecomposable declarative sentences)

Atomic propositions can be connected to each other with logical connectives and compose propositional formulas. The most common connective relations are:

- Negation $\neg$
- Conjunction $\wedge$

- Disunction $\vee$
- Implication $\rightarrow$

The Backus Naur form (BNF) for propositional logic well-formed formula:

$$\phi ::= p \mid (\neg\phi) \mid (\phi \wedge \phi) \mid (\phi \vee \phi) \mid (\phi \rightarrow \phi)$$

where p - is any atomic proposition.

Based on these we can construct a system of reasoning: from some set of formulas we can infer (prove) another formula that can be true or false:

$$\phi_1, \phi_2, \phi 3, ...\phi_n \vdash \psi$$

Such formulas called sequents, where on the left side there is a set of formulas called premises and on the right side - a formula called conclusion

Another relation between logical formulas is semantic entailment:

$$\phi_1, \phi_2, \phi 3, ...\phi_n \models \psi$$

it means that if propositional logic formulas $\phi_1..\phi_n$ evaluate to True, then formula ψ also evaluates to True.

The formula ϕ is valid if for all valuations of atomic propositions in formula it valuates to True:

$$\models \phi$$

SAT solver is a program which aims to solve the Boolean satisfiability problem (SAT), where the Boolean satisfiability problem sometimes called propositional satisfiability asks whether there exists an interpretation that satisfies a given Boolean formula.

Given formula ϕ is **satisfiable** if it has a valuation in which it valuates to True.:

In other words, it asks whether the formula's variables can be consistently replaced by the values TRUE or FALSE to make the formula evaluate to TRUE. If this is the case, the formula is called satisfiable, else unsatisfiable.[25] SAT problem is NP-complete, as a result, only algorithms with exponential worst-case complexity are known. In spite of this, efficient and scalable algorithms for SAT were developed during the 2000s, which have contributed to dramatic advances in the ability to automatically solve problem instances involving tens of thousands of variables and millions of constraints.

SAT solvers usually begin by converting a problem to conjunctive normal form (CNF). **Conjunctive normal form (CNF)** - as a conjunction of clauses, where each clause is a disjunction of literals. Literal is an atomic proposition or a negated atomic proposition. Conjunctive normal form allows easy check of validity of a formula which otherwise take times exponential to number of atoms.

2.1.2 SMT

Propositional logic is limited to express relations between logical components other than equivalent to natural language: *not, and, or, if..then* and declarative sentences that can only be True or False.

In order to be able to operate with finer logical structures we need to enrich the language. The necessity to solve this problem lead to design of Predicate logic or first-order logic.
In First-order logic we introduce:

- Variables that represent individual objects.
- Functions $\mathcal{F}$ that refer to objects.
- Predicates $\mathcal{P}$ - symbols that semantically represents some property or relation.
- Quantifiers: universal $\forall$ and existential $\exists$.

The Backus Naur form (BNF) for predicate logic formulas:

$$\phi ::= P(t_1, t_2, ...t_n) \mid (\neg\phi) \mid (\phi \vee \phi) \mid (\phi \wedge \phi) \mid (\phi \rightarrow \phi) \mid (\forall x\phi) \mid (\exists x\phi)$$

where,
t - called Terms and can be variable, constants or functions on terms

$$t ::= x \mid c \mid f(t, ...t)$$

Atomic formulas are predicates applied to terms: $P(t_1, t_2, ...t_n)$

Variables can be free (supplied from outside the formula) or bounded by some quantifier.

Unlike propositional logic, in predicate logical we cannot just evaluate formulas based on given valuation by assigning True or False to the variables. We require a model of all function and predicate symbols involved.
A model M for a first-order logic contains:

- Domain A is a non-empty set of individuals, mapping each variable to a value in A
- Set of functions where n-ary function is interpreted as a concrete function $f^m : A^n \rightarrow A$.
- Set of predicates n-ary predicate is interpreted as a relation $P^m \subseteq A^n$

Truth in a model is defined recursively:

$$M \vDash P(t_1, ..., t_n) iff (t_1, ..., t_n) \in P^m$$
$$M \vDash \forall x \phi iff for every a \in A, M \vDash [x \to a]\phi$$
$$M \vDash \exists x \phi iff for some a \in A, M \vDash [x \to a]\phi$$

A sentence has no free variables, so its truth value depends solely on M.

We write $\Gamma \vDash \psi$ (semantic entailment) when every model that makes all formulas in Γ true also makes ψ true.

Soundness and completeness ensure $\Gamma \vdash \psi$ (provability) exactly when $\Gamma \vDash \psi$, but the two perspectives complement each other: proofs are constructive, while counter-models establish non-entailment.

2.2 Model checking

Model checking is based on temporal logic. The model in temporal logic described as a system with few states that can change with time. The formula is not statically true or false in a model, as it is in propositional and predicate logic, it can be true in some states and false in others, the static notion is replaced by a dynamic one.

In model checking, the models M are transition systems and the properties ϕ are formulas in temporal logic. To verify that a system satisfies a property, we must do three things:

1. model the system using the description language of a model checker, arriving at a model M;
2. code the property using the specification language of the model checker, resulting in a temporal logic formula ϕ;
3. Run the model checker with inputs M and ϕ.

Model checking is a core approach in Formal verification that involves an algorithmic search of the state space of a system model to check if a given property (specification) holds, where:
Model- is an abstract representation of the system (often a state-transition graph or a program) capturing its possible states and transitions. The model can be described in a modeling language (like Promela for SPIN, or as a transition system in a tool like NuSMV).

Specification- is a formal property that the model should satisfy, commonly expressed in a temporal logic such as LTL or CTL. For example, an LTL specification might state that "whenever request happens, eventually response occurs" or a CTL property might require that "in all execution paths, a certain error state is never reached."

2.2.1 Classification of Model checking methods

Model checking can be classified based on time concept into: **linear-time temporal logic** and **branching time logic.**

Linear-time temporal logic

Linear-time temporal logic (LTL) considers time as a set of paths, where each path is a sequence of time instances. LTL has the following syntax in Backus Naur form:

$$\phi ::= T \mid \ \bot \mid p \mid (\neg\phi) \mid (\phi \wedge \phi) \mid (\phi \vee \phi) \mid (\phi \rightarrow \phi)$$

$$\mid (X\phi) \mid (F\phi) \mid (G\phi) \mid (\phi \ U \ \phi) \mid (\phi \ W \ \phi) \mid (\phi \ R \ \phi)$$

The connectives X, F, G, U, W, R called temporal connectives, because they describe relations between states in different moments of time. The meaning of these connectives described in table:

X	Next	
F	Finally	
G	Globally	
U	Until	
W	Weak until	
R	Release	

In LTL, model can be represented as a transition system [SL16].
Transition system T:

$$T = (S, \overset{a}{\rightarrow}, L, s)$$

,where:
S - set of states, called the space states of T,
$\overset{a}{\rightarrow}$ - transitions (binary relation of states), associated with some action $a \in A$

L - labelling function that reflects states to propositional atoms
s - initial state of T, can be omitted if we want to describe only global structure.
Sometimes if we consider only one action label we can omit labeling transition: $\rightarrow$.
A path π in a transition system T is a finite or infinite sequence of states and (optionally) actions which transform each state into its successor: $s_0 \overset{a_0}{\rightarrow} s_1 \overset{a_1}{\rightarrow} ...$

Trace or computation in a transition system is a finite or infinite sequence of state descriptions along the path: $L(s_0) \overset{a_0}{\rightarrow} L(s_1) \overset{a_1}{\rightarrow} ...$

Branching temporal logic (CTL)

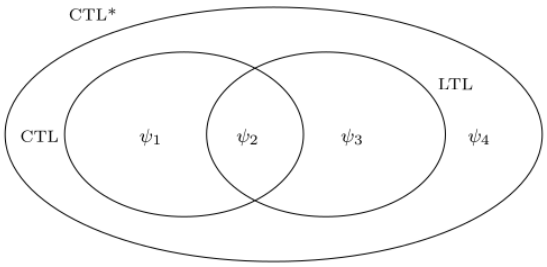
Branching temporal logic represents time as a tree rooted in present and branching into the future, the other name is Computation Tree logic (CTL). LTL implicitly quantifies universally over paths. Therefore, properties which assert the existence of a path cannot be expressed in LTL. We can solve this problem by considering the negation of the property in question. For example to check whether there exists a path from s satisfying the LTL formula ϕ, we check whether all paths satisfy $\neg\phi$; a positive answer to this is a negative answer to our original question, and vice versa. We used this approach when analysing the ferryman puzzle in the previous section. However, properties which mix universal and existential path quantifies cannot in general be model checked using LTL approach, because the complement formula still has a mix. Branching-time logics solve this problem by allowing us to quantify explicitly over paths.

CTL has the following syntax in Backus Naur form:

$$\phi ::= T \mid \bot \mid p \mid (\neg\phi) \mid (\phi \wedge \phi) \mid (\phi \vee \phi) \mid (\phi \rightarrow \phi)$$

$$\mid (AX\phi) \mid (EX\phi) \mid (AF\phi) \mid (EF\phi) \mid (AG\phi) \mid (EG\phi) \mid A[\phi\ U\ \phi] \mid E[\phi\ U\ \phi]$$

New introduced quantifiers A and E mean:
$A\phi$ - formula ϕ for all paths
$E\phi$ - exists a path that formula ϕ
Connectives X,F,G cannot occure without being preceded by A or E.
Weak until W and release R are usually not included in CTL, but can be derived.



Temporal logic (CTL*)

Statistical Model Checking (SMC)

Some systems have huge or continuous state spaces (e.g. complicated analog components, or software with random timing), or they exhibit probabilistic behavior (randomized protocols, hardware failures) where one is interested in quantitative properties (like "the request will be served within 1s with probability at least 0.99"). Statistical Model Checking (SMC) is an approximate verification technique that uses simulation and statistical methods instead of exhaustive exploration [**41**].

SMC treats the model checking problem as a statistical inference problem [**41**] : - It executes a number of simulations (random or guided) of the system. - Checks whether each simulation trace satisfies the property ϕ (which often is a bounded temporal logic property like "within T time units, something happens"). - Based on the outcomes, it uses statistics (like hypothesis testing or estimation with confidence intervals) to infer with high confidence whether the property holds with required probability.

SMC was pioneered by Younes in the 2000s 42 . There are two main modes: hypothesis testing (e.g. "check if the probability $P(property) > p$ with confidence 95%") and estimation (estimate the probability itself within some error bound). Tools like PRISM and its SMC extension, or UPPAAL SMC, or PLASMA Lab implement statistical model checking for various kinds of stochastic systems.

Advantages: The obvious advantage is that SMC sidesteps the state explosion by not exploring the full state space – instead, it samples paths. This makes it scalable to systems with enormous or even infinite state spaces, as long as we can simulate the system. It is also applicable to systems with continuous dynamics or very large numbers of states where traditional model checking is infeasible. Another benefit is ease of implementation: if there is a simulator for your system, adding a statistical checker on top is easier than building a full model checker. SMC naturally supports probabilistic properties – e.g., verifying properties of a Markov Chain or Markov Decision Process

by sampling behaviors and using results like Chernoff or Hoeffding bounds to determine if the property likely holds. Because it uses actual simulations, SMC can handle very complex nonlinear or black-box system aspects (treating them as black boxes that just produce simulation traces).

Disadvantages: The trade-off is that SMC provides a confidence result, not a guarantee. It is probabilistic verification: it might say "with 99% confidence the property holds with probability ≤ 0.97". There's a small chance of error in the verification result (false positives/negatives bounded by the confidence level). Also, SMC may require a large number of simulations to reach high confidence, especially for rare events (if the property failing is a rare event, one might need many samples to observe it even once). Rare property verification is a known challenge – advanced importance sampling or rare event simulation techniques might be needed in such cases.

SMC can't truly prove a property in the mathematical sense – it can only give statistical evidence. If a property almost always holds but has a corner-case failure, SMC might miss it if that scenario has very low probability and isn't sampled. Thus, it sacrifices completeness. Another limitation is that SMC typically handles quantitative properties (often expressed in probabilistic temporal logics like PCTL or BLTL) – it's not usually used for pure logical properties that are either true or false; it's more for properties like "the probability of an event is at least p".

Use Cases: SMC is popular in domains like cyber-physical systems and systems biology, where systems are complex and stochastic. For example, verifying that an automated car controller has less than 10^{-6} probability of collision in a particular scenario could be attacked with SMC by running many simulations of random traffic situations and measuring the frequency of collision. Another area is network protocols with randomness (e.g. randomized consensus algorithms, or the Bluetooth device discovery which has random backoffs) – one can estimate latency or success probability with SMC when exact Markov chain analysis is too complex. Tools such as PRISM can perform both numerical model checking and SMC. PRISM's modeling language allows you to switch to a "sampling mode" for very large models. For instance, PRISM has been used to analyze biochemical reaction networks and one can use SMC when the exact state space of a chemical reaction model (which could be astronomically large) is too big. UPPAAL SMC is another tool, targeting real-time systems with stochastic aspects (like task execution times given as distributions). It allows queries like "what is the probability that the response is below 5ms?" to be answered via simulation. In summary, statistical model checking extends the reach of verification to systems that were previously out of scope, at the cost of absolute certainty. It's a useful approach when classic model checking runs out of steam due to state space size or when dealing with inherently probabilistic systems.

Runtime Verification and Monitoring

Runtime verification (RV) is a technique that involves monitoring a running system (or execution traces produced by it) against a formal specification. Unlike the other methods discussed, which are largely offline (analyzing a model mathematically), runtime verification is applied during execution. It can be seen as a lightweight formal method, sitting somewhere between testing and model checking: you instrument the system with monitors, and these monitors check whether the execution is satisfying the desired properties. If a violation is detected, it can be logged, or in some cases, a recovery action can be taken.

Key components of runtime verification include:

- A set of monitors or runtime assertions derived from the formal specification (often temporal logic formulae or automata on traces). - An instrumentation of the system to report events to the monitors. - The monitors run in parallel with the system (or on logged traces) and detect property satisfaction or violation on the fly.

For example, if you have a property "whenever interrupt is raised, service routine runs within 100ms," a runtime monitor can watch for interrupts and check timing until service routine is called, flagging an error if the 100ms deadline is missed.

Advantages:

Runtime verification avoids the state explosion problem entirely, because it doesn't explore all behaviors — it only checks the actual behaviors that occur. As the Wikipedia definition puts it, "runtime verification avoids the complexity of traditional formal verification techniques (like model checking and theorem proving) by analyzing only one or a few execution traces and by working directly with the actual system" 49. This means it scales well to very complex systems, since you're just running the system (or a high-fidelity simulation of it) and the overhead is only in checking the property on that run. It also has the advantage of checking the real system (no abstraction or modeling error, since the code that runs is what's verified against the property, at least for that run). It can catch errors in scenarios that were not anticipated by designers as soon as they occur in the field or during testing, thus serving as a powerful debugging and validation aid. Runtime verification can be used continuously in deployed systems for critical properties (e.g., a monitor that ensures no unauthorized file access operations occur, and triggers an alarm if they do, providing a layer of security monitoring). Another plus is that runtime verification can sometimes detect issues that static model checking might miss due to environment assumptions. At runtime, the real environment provides inputs and the monitor can see violations under real conditions, including those hard-to-model aspects.

Disadvantages: The fundamental limitation is that runtime verification is not exhaustive. It can only inspect the executions that actually happen (or a sample of them). If a particular bug does not manifest in the observed runs, RV cannot detect it. It provides no guarantee that "no bug exists" — only that "no bug was observed in the runs we monitored". In other words, it lacks completeness/coverage, unlike model checking which (for a model) covers all behaviors by design 49 . Another challenge is performance overhead: monitors consume resources, and if properties are complex (say, specified in LTL, which might require monitoring a suffix of the trace), the runtime checking might slow the system or require buffering events. However, in many cases monitors can be optimized or run on a separate core, and the overhead is manageable (reports of a few percent overhead for simple monitors are common 50). Additionally, writing good monitors (formal properties) and instrumenting the system requires effort. It's easier than full verification, but still requires formal specification skills. If one instrument too much or monitors too many properties, the overhead can become significant, or the system's real-time behavior might be perturbed.

Use Cases: Runtime verification is often used in safety-critical systems as an additional runtime safety net. For instance, in aerospace or automotive software, one might include monitors for conditions like "no two actuators turn on simultaneously" or "if sensor reading exceeds threshold, system enters safe mode within 1 second," etc. NASA has explored RV for spacecraft software monitoring during missions (since you cannot model check the entire flight code with all analog intricacies, but you can monitor key properties as the mission progresses). Runtime Verification has also become popular in security (e.g., monitoring program execution for compliance with security policies) 51 . For example, a monitor can watch system calls to detect a sequence that matches a known attack pattern or a violation of a security automaton. Another domain is testing: during testing, using runtime monitors can greatly increase the chance of noticing an issue. Instead of manually inspecting outputs, formal monitors can rigorously check that each test execution satisfies the expected temporal properties. This is often called "monitoring oracles" in testing – formal specifications serve as test oracles at runtime. In summary, runtime verification does not replace model checking but complements it. Where model checking cannot be applied (due to scale or undecidability), or in deployed scenarios where you want continual assurance, RV provides a way to leverage formal specifications dynamically. It trades completeness for scalability and immediacy.

2.2.2 Comparison of Model checking methods

State explosion - the exponential growth of the state space with system parameters (like number of components or data bits). Combating state explosion is a central theme

in model checking research

2.2.3 Application of Model checking

Hardware model checking

Software model checking

Reactive systems model checking

Reactive system: a system that maintains an ongoing interaction with its environment, responding to inputs with outputs continually (examples include operating systems, web servers, control systems). Reactive systems are typically non-terminating and concurrency-intensive.

2.3 Self-replicating robotic systems

2.3.1 Self-replication introduction

Self-replication is the process by which a dynamical system constructs an identical—or very similar—copy of itself. According to Sipper, replication is an ontogenetic, purely developmental phenomenon: it involves no genetic- or code-modifying operators and yields an exact duplicate of the parent organism.

Replication must therefore be distinguished from reproduction, a phylogenetic, evolutionary process that employs genetic operators (e.g., crossover and mutation in biology) and ultimately generates diversity and evolution [Sip98].

This study focuses exclusively on replication and self-replication, code-altering and evolutionary processes are beyond its scope.

Self-assembly refers to the autonomous organization of components into patterns or structures without human intervention. In robotics, this term may describe either (i) a kinematic machine that manipulates discrete parts to build an assembled copy of itself (kinematic self-replication) or (ii) a collective of mobile robots that can autonomously connect, disconnect, and reconfigure themselves (swarm robotics and robotic self-reconfiguration [Tan+20]).

When the replication process is controlled solely by the system being reproduced, it is called a self-replicating system [LC07]. In contrast, if the replication process requires external elements that actively control and manipulate resources and direct the process, it is simply a replicating system. A biological analogue is a virus, which replicates by exploiting a host cell.

A self-replicating machine or self-replicating robotic system (SRRS) is thus an autonomous robot capable of manufacturing a copy of itself from raw materials, or from minimal possible units related to considered environment, thereby exhibiting self-replication in a manner analogous to that observed in nature.

2.3.2 Classification of Self-replication systems

Type-related replication

Building upon the foundational work of Jones [Jon21] and works of Lee and Chirikjian , self-replicating systems can be systematically classified according to their type-related replication architecture into three primary categories:

Centralized replication system where robots can produce only robots without replication functionality as is shown on Figure 2.1a.
The centralized approach has several distinct advantages: it provides precise control over population growth, ensures quality consistency through standardized manufacturing processes, and reduces resource competition among robots. However it also introduces significant vulnerability through single points of failure - if the replicating robots are damaged or destroyed, the entire system's reproductive capacity is compromised.

Additionally, the concentration of replication resources that have to be located or transported near the replicating robots may create bottlenecks that limit the system's expansion rate and adaptive responsiveness.

Decentralized replication represents a system architecture where every robot within the system possesses full replication capabilities, see Figure 2.1b. This distributed approach is inspired by cellular division processes, where each entity contains the complete blueprint and machinery necessary for creating identical copies of itself.

The decentralized model excels in resilience, scalability and speed of self-replication. The system can continue expanding even if individual robots are eliminated, as each surviving unit has complete reproductive autonomy. This architecture facilitates rapid colonization of new environments and provides redundancy against catastrophic failures. However, decentralized systems face challenges in coordination and control for some systems, possible uncontrolled exponential growth, and consistent quality across independently operating replication processes.

Hierarchical replication architectures represent a hybrid approach that combines elements of both centralized and decentralized strategies. In these systems, robots with replication capabilities can produce two distinct categories of offspring: specialized robots lacking replication ability (similar to centralized systems) and fully autonomous robots capable of self-replication (similar to decentralized systems), as is shown on Figure 2.5.

This multi-tiered approach enables dynamic adaptation to environmental conditions and mission requirements. The system can produce specialized worker robots for specific tasks while simultaneously maintaining a population of replicator robots to ensure continued expansion and redundancy.

The hierarchical model allows for optimized resource allocation, where replication-capable robots can focus on expansion while non-replicating robots specialize in operational tasks, environmental sensing, or resource collection.

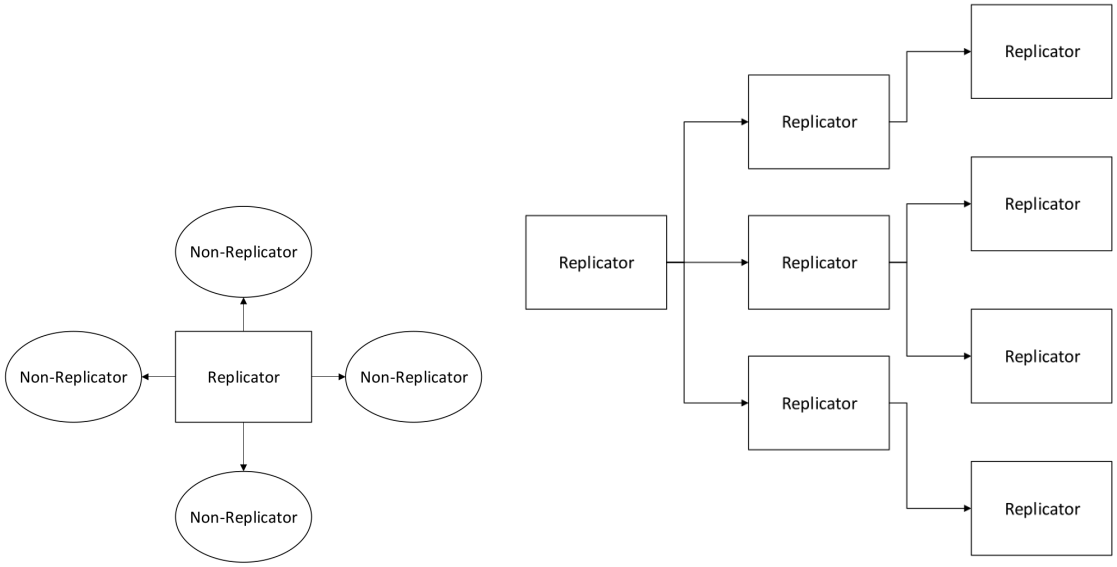


Figure 2.1: Centralized and decentralized replication

Population Composition Classification

The composition of robot types within an SRRS fundamentally influences system behavior, specialization potential, and adaptive capacity. This classification dimension addresses the diversity of robotic types operating within the system.

Homogeneous Systems Homogeneous SRRS consist of robotically identical units, where all entities share the same physical design, computational capabilities, and behavioral programming. This uniformity creates predictable system dynamics and simplifies coordination protocols, as all robots operate with identical sensor suites, actuator capabilities, and decision-making algorithms.

The homogeneous approach offers advantages in replication process consistency, simplified communication protocols, and predictable emergent behaviors. Resource allocation becomes straightforward when all units have identical requirements, and system-wide updates or modifications can be uniformly applied. However, homogeneous systems may struggle with task specialization and environmental adaptation, as the lack of diversity limits the system's ability to optimize for varied operational requirements.

Heterogeneous Systems Heterogeneous SRRS incorporate multiple distinct robot types, each potentially specialized for specific functions, environmental conditions, or operational roles. This diversity enables flexible division of labor, where different

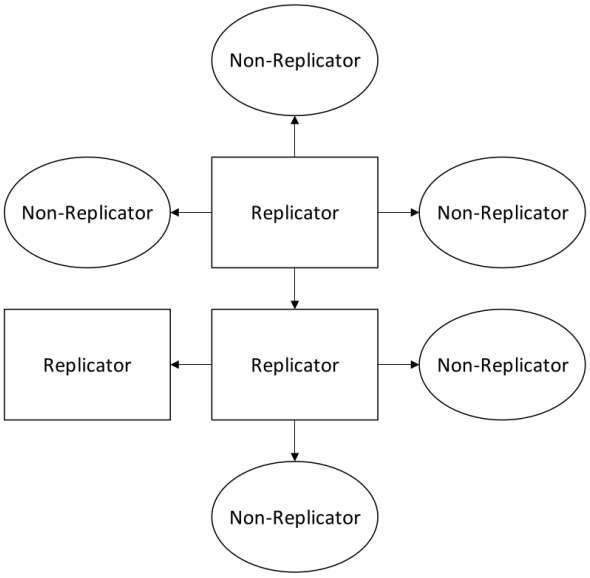


Figure 2.2: Hierarchical replication

robot types can be optimized for exploration, resource gathering, construction, maintenance, or replication tasks.

The heterogeneous model provides enhanced adaptability and operational efficiency through specialization. Different robot types can be designed with complementary capabilities, creating synergistic effects that exceed the performance of individual units. This approach enables the system to respond more effectively to complex environmental challenges and mission requirements.

However, heterogeneous systems require sophisticated coordination mechanisms to manage inter-type communication, resource distribution, and collaborative decision-making across diverse robotic platforms.

Interaction with environment or external system

Another classification of self-replicating robotic systems can be made based on the degree of autonomy and reliance on external systems, as it described in [LC07]. A fundamental distinction is made between passive and active self-replication:

Passive self-replicating systems do not possess inherent mechanisms to actively reproduce themselves. However, given an appropriate environment and the right configuration of components, replication can still occur. A canonical example of this is

the Penrose block replicators, which demonstrate self-replication through mechanical constraints and spatial arrangement without any active control [Pen58].

Active self-replicating systems, on the other hand, are equipped with the functional capacity to carry out the replication process. These systems may still rely on passive environmental structures (such as fixtures or gravity) to assist in replication, but are actively engaged in constructing replicas of themselves.

Interaction with environment or external system, alternative classification

A more detailed and operationally useful classification is introduced by Suthakorn, Zhou, and Chirikjian [SZC02]. This framework emphasizes how SRRS interact with their environments and whether replication is accomplished directly or through intermediate stages. Based on this framework, all SRRS are categorized into two primary classes:

Directly replicating SRRS - in which a robot is capable of constructing an exact copy of itself in a single generational step. These systems can be further divided into the following subcategories, based on their reliance on fixtures and replication strategy:

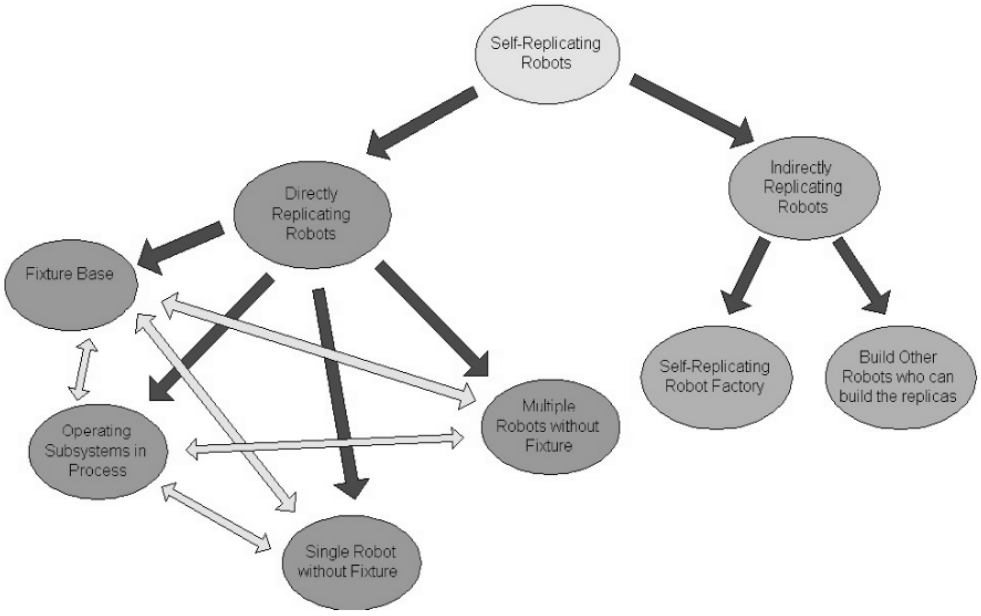


Figure 2.3: Directly and indirectly replicating robots classification

Directly Replicating Robots can be further classified into following groups:

- Fixture-Based Group The self-replicating robots in this group depend on external fixtures in order to complete the self-replication process. Passive fixtures are able to assist in this because of the shape constraints that they impose. In some other cases, to unify subsystems, push-pull fixtures are helpful as well.
- Operating-Subsystem-in-Process Group In this group one or several subsystems of the replica can operate before the replica itself is fully assembled. These subsystems are able to assist the original self-replicating robot during the assembly of the replica.
- Single-Robot-Without-Fixture Group In this group only one robot is used to finish the self-replication process. Thus, the robot in this group depends only on the available environment. Usually, the complexity of the subsystems or the number of subsystems in the replica is very low for this group.
- Multi-Robot-Without-Fixture Group In this group more than one robot works together in the self-replication process without the assistance of fixtures. A major advantage is the reduction of the time required for self-replication. A disadvantage is that there may be interference problems among robots. There are several possible ways that a self-replicating robot can be categorized in two or three groups mentioned above. The combination of two or three different concepts can be incorporated in a potential design, such as a combining operating-subsystems-in-process with fixture-based robots. More categories are likely to be developed in the subsequent stages of our research in the area of self-replicating robots.

Indirectly replicating - where replication is achieved through intermediate stages rather than a direct one-to-one duplication. The original robot initiates the creation of other robots or constructs a specialized environment that facilitates replication. Indirect replication strategies include:

- Self-Replicating Robot Factory A robot or group of robots constructs a fabrication environment — a factory, that can then autonomously produce replicas of the original. This approach encapsulates the replication process into a fixed infrastructure, enabling high throughput and modular production.
- Intermediate Robot Builders Instead of replicating itself directly, a robot creates new robots that are not replicas, but are instead specialized for manufacturing. These intermediate agents then produce replicas of the original robot. This cascading replication strategy introduces flexibility and specialization but may involve increased system complexity.

2.3.3 Self-replication models

The concept of self-replicating systems originates from the foundational work of Neumann and Burks, who in 1962 proposed the first theoretical model of a self-reproducing automaton [NB62]. This model, developed in the context of cellular automata, formalized the minimal components necessary for a machine to replicate itself.

According to this model, a self-reproducing automaton consists of four key components:
A (constructor): an automatic factory that collects raw materials and processes them into outputs using a specified written instruction,
B (copier): that takes an instruction and copies it,
C (controller): that controls A and B, actuating them alternatively,
D (instructions) A coded description of the automaton's structure and function, acting as its "genetic" information.

$$(A + B + C + D) \rightarrow (A + B + C + D), (A + B + C + D)$$

And the replication process in von Neumann's model, is when the automaton $(A + B + C + D)$ produces another $(A + B + C + D)$.
However, while von Neumann's model is conceptually complete for information-theoretic or logical systems, it exhibits limitations when applied to physical robotic systems. Specifically, it doesnt describe interaction with a physical environment, resource acquisition, resource transformation and management of by-products, energy, and mechanical constraints.

To address these limitations, a more general and physically grounded model was proposed by Lee and Chirikjian [LC07]. In this extended model, the replication process is formalized as a transformation: Lee and Chirikjian proposed broader model, where a replication process can be written as

$$(R, M, E) \rightarrow (R', M', E') \tag{2.1}$$

where:
M is a multiset of available parts to be used for building replicas by an initial system
R is a multiset of an initial functional system to be reproduced. $R \neq 0$; to be a replicating system.
E is a multiset of environmental structures and/or elements involved in the replication process (but not replicated).
R', M', E', the replicated entities, the remaining or altered material and the possibly changed environment respectively.

In this model R describes the whole replicator $A + B + C + D$ from Von Neumann's model.

This transformation can also be expressed functionally as:

$$(R', M', E') = Phi(R, M, E, t) \tag{2.2}$$

Where Φ denotes the replication function and t is time. This formulation enables modeling dynamic replication scenarios over time and accounts for state changes in materials and environment.

The model explicitly incorporates environmental interaction (E), which plays a critical role in physical self-replication. As proposed by Eno, Mace, Liu, et al. [EML+07], environments can be categorized based on the level of structure and predictability:

- Completely structured environment - in which every environmental structure is fixed at a precise location and there is no modification or change in any environmental structures during the self-replication process. If we define E' is the set of environmental structures after self-replication process: $E = E' \neq \phi$
- Partially structured environment is like a structured environment, but with structures whose locations can be permuted and perturbed by a small random error in pose during the self-replication process. $E \neq E \neq \phi$
- Unstructured environment has no predefined or organized structure and can dynamically change during the self-replication. Can be formally depicted as: $E = \phi$ where ϕ represents a stochastic or unknown environmental process.

2.3.4 Degree of replication

In 2007 Lee and Chirikjian proposed a measure of degree of replication [LC07]: measure of how much order is created by the assembly process relative to the existing order in the unassembled parts. In this section, we first define a combined measure of system complexity distribution and relative complexity for ARS.

As a simple measure of subsystem complexity, we count the number of active elements in each subsystem. An active element is a moving mechanical part or a fundamental electronic component. Some special parts, such as a chassis and fixed mechanical parts with special purpose (e.g., a passive end- effector), are also viewed as active elements even if they are not moving mechanical parts.

For a robotic system (recall that all robotic systems are active by our definition) consisting of n subsystems, we define the degree of replication for the system, D_s , as

$$D_s = \frac{C_{min}}{C_{max}} \cdot \frac{C_{total}}{C_{ave}} \cdot \frac{1}{C_{ave}} \tag{2.3}$$

where

$$C_{ave} = \frac{1}{n} \sum_{i=1}^{n} C_i$$

$$C_{max} = max C_1, ..., C_n$$

$$C_{min} = min C_1, ..., C_n$$

$$C_{total} = \sum_{i=1}^{n} C_i + \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} C_{ij}$$

C_i is the number of active elements in M_i
C_{ij} is the number of interconnections between the i-th and j-th subsystems when they are assembled. $C_{ij} = 0$ if the i-th and j-th subsystems are not directly connected by the assembly process.

In formula 2.1 the first term measures the complexity distribution throughout the subsystems, the second term measures the relative complexity of the total system to the individual subsystems, and the last term penalizes for complex subsystems. To design a robotic system capable of replication from low-level parts, the subsystem complexity should remain low relative to the total system complexity (i.e., high relative complexity).

In addition, we view a system consisting of a few complex parts as more trivial than one composed of many low-complexity parts. A higher value of D_s indicates a more truly replicating system in terms of its complexity distribution and relative complexity. Although the concept of D_s may not generalize to all robotic systems, it is useful as a measure to evaluate robotic replicating systems.

2.3.5 Implementations of kinematic self-replicating robotic systems

Low-complexity 2D assembly systems

The experimental system developed by Lee and Chirikjian demonstrates active self-replication using six simple electromechanical subsystems [LC07]. The system operates autonomously within a partially structured environment and constructs a functional replica of itself without external control or manipulation. This prototype serves

as a proof-of-concept for scalable robotic self-replication using low-complexity parts and embedded environmental cues.

The system is a physical prototype of a self-replicating robot made up of six simple electromechanical subsystems (called M1 to M6) as it shown on the Figure 2.4.

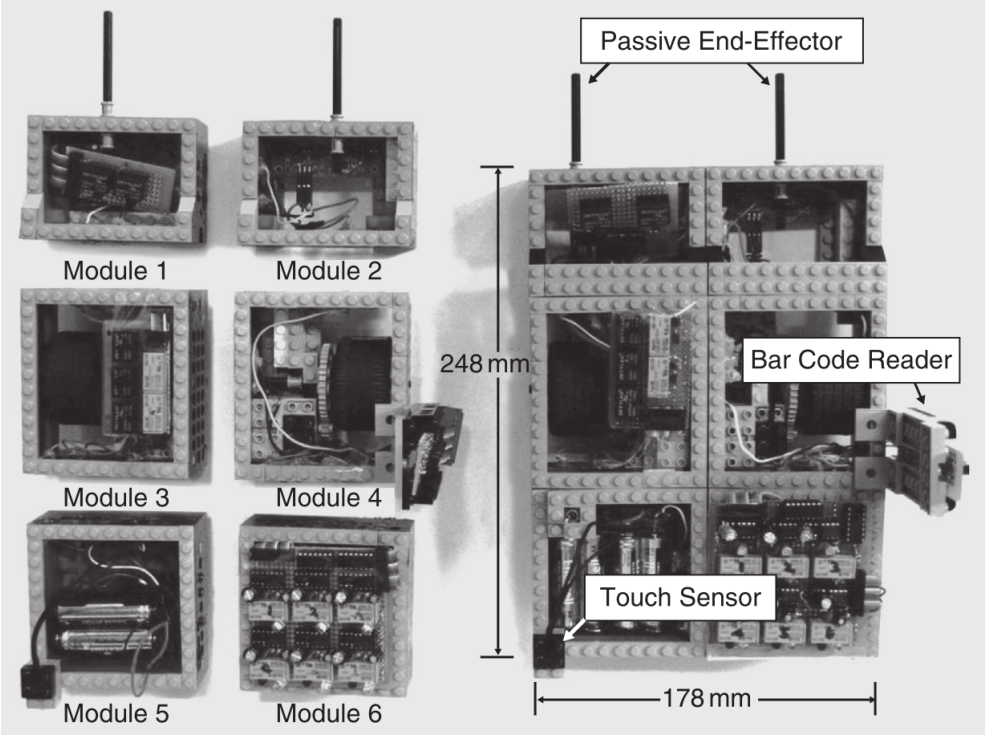


Figure 2.4: Modules (left) and complete robot (right)

These modules, when assembled in a specific order, form a complete and functional mobile robot. The purpose of the experiment was to demonstrate that robotic self-replication is possible using relatively low-complexity parts in a partially structured environment—a space that aids replication without directly controlling it. Together, these modules allow the robot to: move along predefined paths, detect locations of components, decide what action to perform and carry and assemble parts.

The robot operates within a partially structured environment—one that contains helpful features, but does not actively participate in the replication. It includes: outer and inner tracks, crossways (branch paths), bar codes, contact codes, sensors, assembly station and wall, used to align the robot during reversing, ensuring precise placement.

The robot has six states, see Figure 2.6 and three distinctive behaviors for each state: forward line-tracking (line-tracking mode), reversing (reversing mode), and turning left while the timer is on (left-turning mode). The replication process begins with

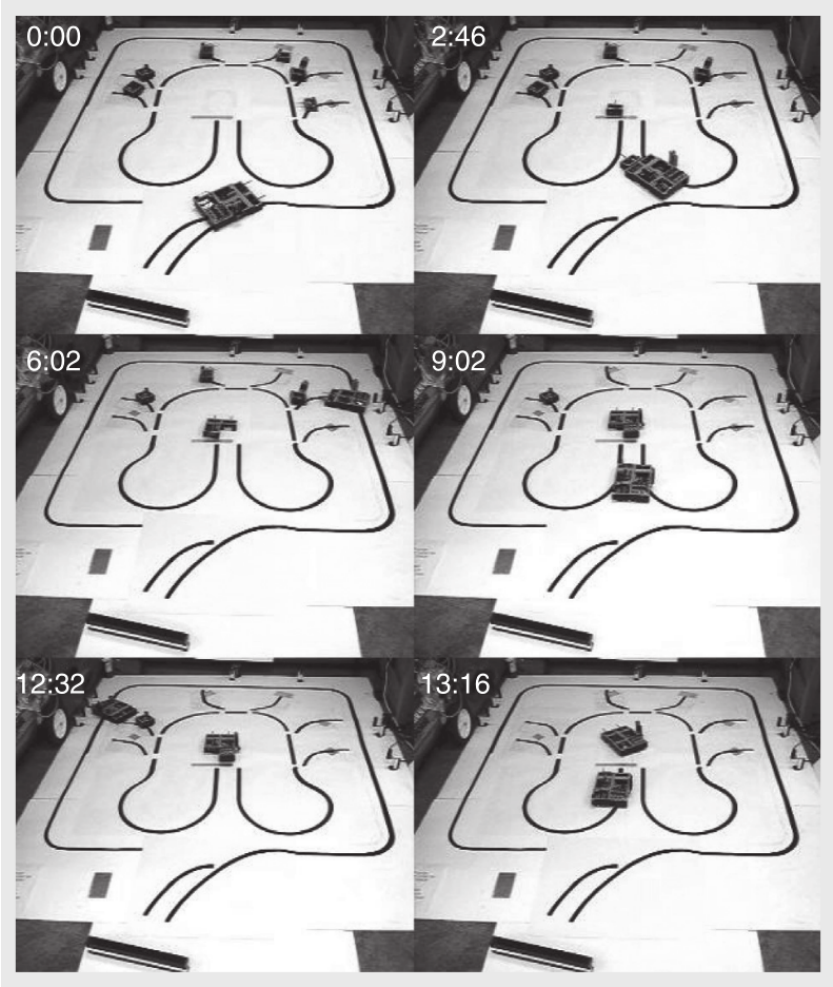


Figure 2.5: The process of self-replication

the parent robot navigating along a black track using its line-tracking sensors. As it moves, it detects barcodes placed near the modules it needs to collect; each barcode signals the robot to turn and approach a module. Using its passive end-effector, the robot captures the module and transitions to a new state based on contact sensors. It then transports the collected module to a central assembly station marked by a metal line, which triggers the robot to reverse. The robot backs into position and places the module, using a wall as a physical guide for alignment. After delivering the module, it returns to the track and continues this process for the remaining modules. Each time, a new barcode guides it to the next module, and contact codes confirm successful transitions. The modules are assembled in a layered structure, with specific modules forming the base, middle, and top of the robot. Once all six modules are in place and properly connected, the electrical circuit is completed, and the new robot becomes

fully functional. The entire process is autonomous and guided solely by the robot's internal logic and environmental cues successfully constructing a working copy of itself without any external control or assistance.

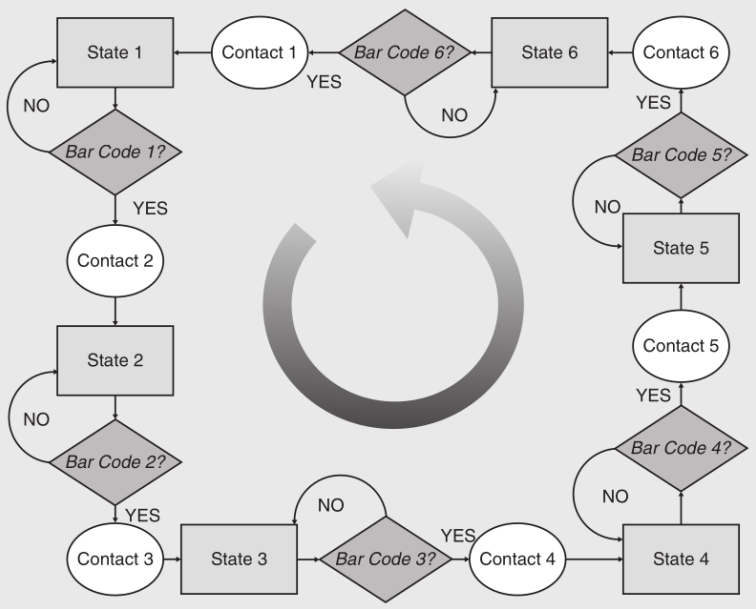


Figure 2.6: State diagram shows behaviors of the robot according to the sensor inputs

Using formalas 2.1 and 2.2 the system can be represented by

$$R = M_1 + ... + M_6$$

$$M = M_1 + ... + M_6$$

$$E = \{\text{tracks, bar codes, contact codes, wall, metal line}\}$$

, the system becomes

$$R' = \{(M_1 + ... + M_6),\ (M_1 + ... + M_6)\}$$

$$M' = \emptyset$$

$$E' = E.$$

Molecubes

Material-robot systems

The material-robotic system proposed by Jenett et al. in [Jen+19] and further developed by Abdel-Rahman et al. in [Abd+22] consist of passive structural lattice voxels which

form a substrate for the locomotion of purpose-built inchworm modular robots build from active voxels, and capable of rearranging and placing additional voxels.

The combination of voxels and robots forms a system, enabling precise assembly of large structures with simple robots through localized registration to the underlying lattice, and also allows of creation of modular robotic toolkit that utilizes active lattice voxels as the primary structural building blocks.

The active voxels which form the basis of the structural-robotic system are a cuboctahedral (Cuboct) unit cell which offers superior structural properties such as high stiffness at low density with geometric characteristics suited for robotic assembly and fabrication. Specifically the fllat Cuboct faces enable fully- constrained connections between single-voxel pairs with four points of contact, as well as a useful decomposition for voxel fabrication.

(a) Cell **(b)** Modular robot

Figure 2.7: Material-robot system

Combining these active voxels with actuators, control, and power enables unique behaviors including robotic self-replication and hierarchical robot assembly. A single robot, tasked with constructing a large structure, can create a heterogeneous swarm of constructors tailored to maximize material throughput for a given target geometry.

One of possible variations of assembler robots, designed to carry voxel cubes, and a process of self-replication from a set of modules is shown in Figure 2.8.

To perform self-replication, an initial “parent” robot uses its manipulators to pick and place voxels from a nearby pickup station. The parent robot builds a copy of

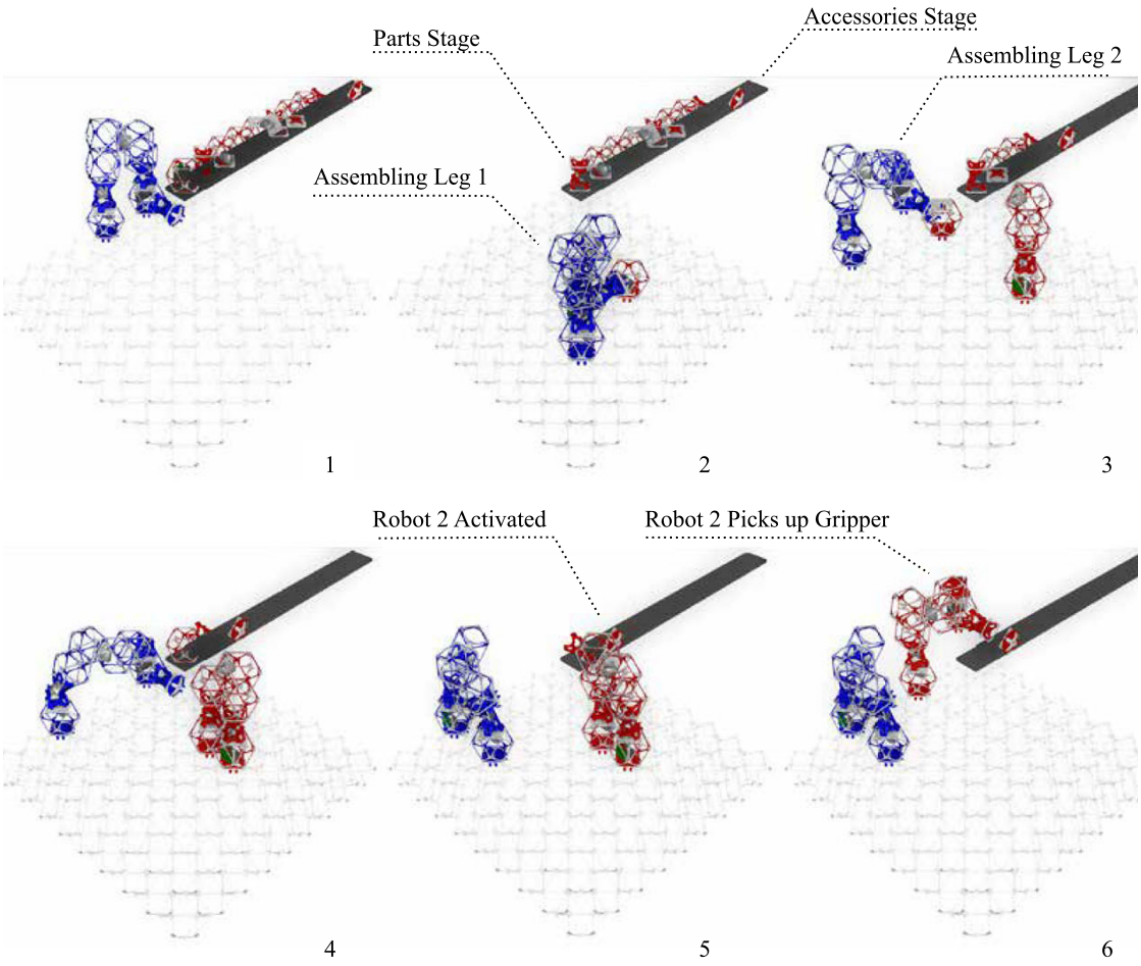


Figure 2.8: Self-replication process

itself—termed a "child" robot—by assembling voxels in a configuration that mirrors its own structure. The replication process is limited to voxels and certain key functional modules, including control units, power supplies, and actuated joints (called elbows). Components like wrists and grippers are added afterward through a procedure called "accessorizing," where the robot attaches these parts to the pre-assembled structure.

A critical constraint in the replication process is that the newly built robot must be free to move upon completion. Therefore, construction cannot proceed directly on the substrate base; instead, it extends out from an anchored platform, ensuring that once finished, the child robot can undock and function independently. The replication logic is programmed into a centralized control system, which assigns tasks, manages timing, and ensures robots do not collide. The control system guides each robot step-by-step in constructing a new robot, allowing for recursive reproduction over multiple

generations.

This system's significance lies in its ability to scale construction tasks by increasing the number of robots through self-replication, rather than requiring the deployment of additional robots externally. As more robots are built, the workload is distributed, theoretically allowing exponential speed-up in large-scale assembly projects. The practical implementation uses inchworm-style locomotion, where robots move stepwise over the voxel lattice using coordinated grippers and joints, making them lightweight, simple, and adaptable to modular construction environments.

While this model is a proof-of-concept, it successfully demonstrates robotic self-replication using standardized parts and relatively low-complexity hardware. The modularity and discrete assembly approach also enhance the system's potential for error detection, robustness, and reconfiguration. Although scalability is limited by actuator strength and coordination complexity, the framework opens new avenues for autonomous robotic manufacturing systems that can grow their own workforce from a finite resource pool.

2.3.6 Research tools for SRRS

3 Research Questions

This paper will address the following research questions:

1. How can automated model checking and other formal-verification techniques be used for verifying liveness (replication reliability) and safety properties in self-replicating robotic systems(SRRS), and what domain-specific adaptations are required for practical deployment?
2. What systematic methodology can translate the behaviours and structural constraints of self-replicating robots into expressive temporal-logic specifications?
3. What fundamental limitations—computational, semantic, or modelling-related—do current face when applied to highly dynamic, self-replicating robotic collectives, and how might these limitations be mitigated?

3.1 Approach

Experimental platform

As a subject of a research a simple robot will be taken, with two grippers and an elbow joint that can manipulate blocks (cuboctahedral voxels) one at a time. An experimental platform will be realised as a simulation in Gazebo environment with control from ROS2 nodes. The blocks and environment geometry robot joints and sensors and conections will be defined in Xacro/URDF files.

Generation of a model and specifications for formal-verification

A custom program has to be written in Python, that parses the Xacro files and ROS2 Nodes controller to generate a NuSMV model:

- a Boolean occupancy matrix representing the neighbourhood on a field relevant to the current replication.

- replication program-counter plus local flags (gripper-occupied, block -held, etc.).

The liveness and safety properties has to be defined independent from the simulation, in a form close to the following:

- Liveness. $G(start \rightarrow F(Robot_1 == ready \wedge Robot_2 == ready))$ — once replication starts, eventually both robots are able to function independently.
- Collision-free safety. $G\neg collision$ (or $AG\neg collision$ for CTL).

Bibliography

[Pen58] L. S. Penrose. “Mechanics of Self-Reproduction”. In: *Annals of Human Genetics* 23.1 (Nov. 1958), pp. 59–72. DOI: 10.1111/j.1469-1809.1958.tb01442.x (cit. on p. 18).

[NB62] John von Neumann and Arthur W. Burks. *Theory of Self-Reproducing Automata*. Edited and completed by Arthur W. Burks. Urbana, IL: University of Illinois Press, 1962 (cit. on p. 20).

[Sip98] Moshe Sipper. “Fifty Years of Research on Self-Replication: An Overview”. In: *Artificial Life* 4.3 (1998), pp. 237–257. DOI: 10.1162/106454698568576 (cit. on p. 14).

[SZC02] Jackrit Suthakorn, Yu Zhou, and Gregory S. Chirikjian. “Self-Replicating Robots for Space Utilization”. In: (2002) (cit. on p. 18).

[Mic04] Mark Ryan Michael Huth. *Logic in computer science Modelling and Reasoning about Systems*. 2nd. Cambridge, New York, Melbourne, Madrid, Cape Town, Singapore, São Paulo: Cambridge University Press, 2004 (cit. on p. 2).

[EML+07] Steven Eno, Lauren Mace, Jianyi Liu, et al. “Robotic Self-Replication in a Structured Environment without Computer Control”. In: *Proceedings of the 2007 IEEE International Conference on Robotics and Automation (ICRA)*. 2007 (cit. on p. 21).

[LC07] Kiju Lee and Gregory S. Chirikjian. “Robotic Self-Replication: A Descriptive Framework and a Physical Demonstration from Low-Complexity Parts”. In: *IEEE Robotics & Automation Magazine* 14.4 (2007), pp. 34–43. DOI: 10.1109/MRA.2007.910 (cit. on pp. 14, 17, 20–22).

[Bec+13] Stephan Beck et al. “Immersive Group-to-Group Telepresence”. In: *IEEE Trans. Vis. Comput. Graph.* 19.4 (2013), pp. 616–625. DOI: 10.1109/TVCG.2013.33 (cit. on p. 1).

[SL16] Valentin Goranko Stéphane Demri and Martin Lange. *Temporal Logics in Computer Science Finite-State Systems*. 2nd. University Printing House, Cambridge CB2 8BS, United Kingdom: Cambridge University Press, 2016 (cit. on p. 7).

[Jen+19] Benjamin Jenett et al. “Material–Robot System for Assembly of Discrete Cellular Structures”. In: *IEEE Robotics and Automation Letters* 4.4 (2019), pp. 4019–4026. DOI: 10.1109/LRA.2019.2930486 (cit. on p. 25).

[Tan+20] Ning Tan et al. “A Framework for Taxonomy and Evaluation of Self-Reconfigurable Robotic Systems”. In: *IEEE Access* 8 (2020), pp. 13969–13986. DOI: 10.1109/ACCESS.2020.2965327 (cit. on p. 14).

[Jon21] Andrew Burkhard Jones. “Decision-Making for Self-Replicating 3D Printed Robot Systems”. Ph.D. Dissertation. North Dakota State University, 2021 (cit. on p. 14).

[Abd+22] Amira Abdel-Rahman et al. “Self-replicating hierarchical modular robotic swarms”. In: *Communications Engineering* 1 (2022), p. 35. DOI: 10.1038/s44172-022-00034-3 (cit. on p. 25).

[25] *Boolean satisfiability problem*. Accessed: 2025-02-31. 2025. URL: https://en.wikipedia.org/wiki/Boolean_satisfiability_problem (cit. on p. 4).

A Appendix

This was just missing.